

1. FEJEZET Java 17 és 21 (LTS verziók)

Mi ez a valóságban? A Java egy olyan programozási nyelv és egyben egy futtatókörnyezet (Virtual Machine), amelyen a szerveroldali szoftverünk lefut. Képzeld el úgy, mint egy gigantikus, szigorú rendszabályok szerint működő virtuális gyárat: te megírod a tervrajzokat (a Java kódot), a Java pedig hibátlanul, a nap 24 órájában levezényli a termelést a számítógép hardverén.

Miért ezt használjuk a projektben? A 17-es és 21-es verziók úgynevezett LTS (Long-Term Support) kiadások. Ez azt jelenti, hogy a készítőк hosszú éveken át garantálják a biztonsági frissítéseiket és a hibajavításokat. Banki, fizetési és kritikus infrastruktúráknál amilyen egy autótöltő hálózat is - nem szabad kockáztatni fapados, instabil technológiákkal.

Hogyan kapcsolódik a többiekhez? A Java maga az alapszerkezet. Ezen rajzolódnak ki a Spring Boot keretrendszer építőkövei, ezen futnak az adatbázis-lekérdezések, és ezen keresztül kommunikálunk a töltőoszlopokkal is.

2. FEJEZET Spring Boot 3.2.4

Mi ez a valóságban? A Spring Boot egy olyan előre megpakolt szerszámoszláda a Java nyelvhez, amely leveszi a fejlesztők válláról a legunalmasabb, ismétlődő feladatokat. Ha tisztán Javában kéne felhúzni egy szervert, ezerszáz sornyi bonyolult beállítást kellene kézzel írni. A Spring Boot ezzel szemben azt mondja: „Figyelj, én már összeraktam neked a szerver motort, csak indítsd el!”

Miért ezt használjuk a projektben? A 3.2.4-es verzió a legfrissebb enterprise szabványokat hozza el. Beépítve tartalmazza a webes szervert (Tomcat), intelligensen kezeli a memóriát, és villámgyorsan összeköti a programunkat az adatbázissal anélkül, hogy nekünk az alacsony szintű hálózati részletekkel kellene bajlódnunk.

Hogyan kapcsolódik a többiekhez? A Spring Boot a vezérlőközpont. Ez fogja össze az 1. fejlesztő adatbázisát, a 2. fejlesztő WebSocket kommunikációját, a 3. fejlesztő üzleti logikáját és a 4. fejlesztő fizetési kapuját egyetlen egységes, futtatható rendszerré.

3. FEJEZET Spring Web (REST API Végpontok)

Mi ez a valóságban? A REST API a szoftverek közötti közös nyelv. Amikor a mobiltelefonodon megnyitod a töltőalkalmazást, és rákattintasz a „Töltés indítása” gombra, a telefonod elküld egy szabványos HTTP üzenetet a szervernek. A Spring Web modul felelős azért, hogy ezt az üzenetet a szerver megértse, feldolgozza, és válaszoljon rá (pl. JSON formátumban).

Miért kritikus a projektben? A külső világ (felhasználók mobilappjai, webes admin felületek, fizetési szolgáltatók) kizárólag ezen a kapun keresztül tud kommunikálni a rendszerünkkel. Ha a Spring Web elromolna, a felhasználó süket fülekre találna a telefonján.

Hogyan kapcsolódik a többiekhez? A 4. fejlesztő dolgozik leginkább ezen a rétegen. Ő írja meg azokat a végpontokat, amelyek fogadják a mobilapp kéréseit, és továbbítják azokat a 3. fejlesztő üzleti logikájának.

4. FEJEZET Spring WebSocket (Kétirányú Csatlakozás)

Mi ez a valóságban? A hagyományos web (HTTP) olyan, mint egy levélváltás: küldesz egy kérdést, vársz, megjön a válasz, és a kapcsolat lezárul. Ezzel szemben a WebSocket olyan, mint egy telefonhívás: a vonal folyamatosan nyitva maradhat órákig is, és mindkét fél (szerver és töltőoszlop) azonnal beszélni tud a másikhoz anélkül, hogy újra kellene tárcsázni.

Miért kritikus a projektben? Az utcán álló elektromos töltőoszlopoknak és a mi szerverünknek folyamatosan, élőben kell kommunikálniuk. Ha a töltőoszlop leejti a kábelt, vagy áramszünet van, azt nem a mobilappnak kell percenként kérdezgetnie, hanem a töltőoszlopnak azonnal szólnia kell a szervernek a WebSocketen keresztül.

Hogyan kapcsolódik a többiekhez? A 2. fejlesztő legfőbb feladata ennek a WebSocket szervernek a felhúzása és a töltőktől jövő üzenetek (pl. BootNotification) feldolgozása.

5. FEJEZET Spring Data JPA (Objektum-Relációs Híd)

Mi ez a valóságban? A programozók Java objektumokkal dolgoznak a kódban (pl. ChargePoint osztály), az adatbázisban viszont sorok és táblák vannak. A Spring Data JPA egy olyan mágikus fordító, amely automatikusan átülteti a Java objektumokat SQL adatbázis sorokká, és fordítva. Nem kell kézzel SQL parancsokat írogatnod a mentéshez.

Miért kritikus a projektben? Minden egyes töltési munkamenetet, felhasznált kilowattórát, pénzüsszeget és töltőstátuszt tartósan le kell menteni. Ha ez a híd elromlik, az adatok elvesznek a memóriából a szerver újraindításakor.

Hogyan kapcsolódik a többiekhez? Az 1. fejlesztő építi ki ezeket az entitásokat és repositorykat, amiket aztán a többi fejlesztő (pl. a státuszváltásoknál vagy a számlázásnál) folyamatosan hívogat az adatok lekérésére vagy mentésére.

6. FEJEZET PostgreSQL (Relációs Adatbázis)

Mi ez a valóságban? A PostgreSQL egy rendkívül okos, táblázatos formátumú digitális raktár. Itt nem egy nagy halom dosszié áll egymáson, hanem szigorú oszlopokra és sorokra osztott adattáblák (pl. külön táblája van a töltőknek és külön a munkameneteknek), amelyek pontosan hivatkoznak egymásra.

Miért kritikus a projektben? Pénzről és energiáról beszélünk. Ha valaki elindít egy töltést, az adatbázisnak szigorú ACID garanciákat kell adnia: nem történhet meg, hogy a pénz levonódik, de a rendszer elfelejti rögzíteni a töltést. A PostgreSQL ezen a téren az egyik legmegbízhatóbb rendszer a világon.

Hogyan kapcsolódik a többiekhez? A rendszer háttérében fut helyben vagy a felhőben, és a Spring Data JPA-n keresztül folyamatosan írják-olvassák az adatok a többi modulból.

7. FEJEZET HikariCP Connection Pool (Kapcsolatkezelő)

Mi ez a valóságban? Az adatbázishoz való csatlakozás olyan, mint hívást indítani egy ügyfélszolgálaton: ha minden egyes adatmentéskor újat hívsz, az rengeteg időbe és erőforrásba telik. A HikariCP előre létrehoz egy „készletű csapatot” (pl. 10 nyitott adatbázis-kapcsolatot), és ha a programnak adatra van szüksége, azonnal átadja az egyiket a készletből, majd visszaveszi.

Miért kritikus a projektben? Ha egyszerre száz autós indít töltést, és a szerver minden egyes kattintásnál nulláról kezdi felépíteni az adatbázis-kapcsolatot, a rendszer azonnal összeomlana a túlzott terheléstől. A HikariCP garantálja a villámgyors válaszidőt.

Hogyan kapcsolódik a többiekhez? Az 1. fejlesztő konfigurálja be a Spring Boot application.yml fájljában, hogy a háttérben működjön, és kiszolgálja a JPA kéréseit.

8. FEJEZET OCPP 1.6-J (Open Charge Point Protocol)

Mi ez a valóságban? Az OCPP az elektromos autós világ univerzális „eszperantó” nyelve. Teljesen mindegy, hogy a töltőoszlopot az ABB, a Siemens vagy a Schneider gyárból hozták, mindegyik ugyanazt a protokoll nyelvet beszéli. Az 1.6-J változat azt jelenti, hogy a szabályrendszer JSON formátumú üzeneteket használ WebSocketen keresztül.

Miért kritikus a projektben? Enélkül a szabvány nélkül kaotikus lenne a helyzet: minden egyes gyártó töltőjéhez külön szoftvert kellene írunk. Az OCPP-nek köszönhetően a töltőoszlop bekapcsoláskor bejelentkezik (Boot Notification), jelzi az állapotát (StatusNotification), és elindítja a tranzakciót.

Hogyan kapcsolódik a többiekhez? A 2. fejlesztő dolgozik ezzel közvetlenül a WebSocket rétegen, feldolgozva a töltőktől érkező OCPP csomagokat.

9. FEJEZET Zero-Risk Tranzakciókezelés & Self-Healing

Mi ez a valóságban? Ez a platformunk legfontosabb fegyvere a piacon lévő többi hibás rendszer ellen. A piacon gyakori, hogy a kútfej rossz kommunikációja miatt zárol a bank 10 000 Ft-ot, de a töltés nem indul el (beragadt pre-auth). A mi rendszerünkben ez nem fordulhat elő:

- **Atomic State Check:** A banki pénz zárolás (pre-auth) csak akkor történik meg, amikor a fizikai töltőoszlop WebSocketen igazolja, hogy a kábel stabilan bent van és készen áll.
- **Automatic Refund Scheduler:** Ha 30 másodpercen belül nincs aktív töltés, a Spring Boot háttér folyamat azonnal küld egy Void parancsot a banknak a pénz felszabadítására.
- **Self-Healing:** Az öngyógyító háttérszálak folyamatosan takarítják az árva munkameneteket (*orphan sessions*).

10. FEJEZET Git & GitHub (Dev Branch Munkamódszertan)

Mi ez a valóságban? A Git egy olyan időgép és dokumentumkezelő, amivel a 4 fejlesztő egyszerre dolgozhat ugyanazon a projekten anélkül, hogy elrontanák egymás munkáját. A GitHub pedig a felhős központ, ahol a közös kód tárolódik.

Hogyan használjuk a projektben? Van egy közös dev (fejlesztési) ág. Mind a 4 fejlesztő lehúzza a kódot a saját gépére: az 1-es fejlesztő az adatbázist írja, a 2-es a WebSocketet, a 3-as az üzleti logikát, a 4-es a REST API-t. Amikor egy modul kész, beküldik a központba, ahol a rendszer összefésüli őket.

Miért kritikus? Enélkül a csapatmunka kaotikus lenne (pl. valaki felülrná a másik kódját egy e-mailben küldött zip-fájllal). Így minden változás nyomon követhető és biztonságos.